Name: Gargi Devendra Shintre
Div: D15C **Roll No.** 44

Experiment No. 1

Aim:

To implement Linear and Logistic Regression on real-world datasets.

Theory:

Dataset Link: Students Performance in Exams

Linear Regression

Dataset Description:
This dataset consists of the marks secured by students in various subjects alongside their demographic and background information. It is used for regression analysis to understand how personal and socio-economic factors influence academic performance.

- **Records:** 1000
- **Target**: math score (Numerical)
- **Features:** Gender, race/ethnicity, parental level of education, lunch, test preparation course, reading score, writing score

Variable	Description	Data Type
gender	Sex of students (male/female)	Categorical
race/ethnicity	Ethnicity of students (Group A, B, C, D, E)	Categorical
parental level of education	Parents' final education level	Categorical
lunch	Type of lunch (standard/free/reduced)	Categorical
test preparation course	Completion of test prep course	Categorical
math score	Score in math (Target)	Numeric (0-100)
reading score	Score in reading	Numeric (0-100)
writing score	Score in writing	Numeric (0-100)

Linear regression is a fundamental statistical method used to model the linear relationship between a dependent variable and one or more independent variables by fitting a straight line to the data, minimizing the sum of squared differences between observed and predicted values. It assumes linearity, independence of errors, homoscedasticity, and normality of residuals.

Mathematical Formula:

In this experiment, a Multiple Linear Regression model is applied to predict math score. The general equation is:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Where y is the predicted math score, xn are the input features (e.g., reading score, lunch type), B0 is the intercept, and Bn are the learned coefficients. The model minimizes the Mean Squared Error (MSE):

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Logistic Regression

Dataset Description:

Using the same dataset, we create a binary classification problem.

- Objective: Predict if a student Passed Math (Score > =50).
- Target: passed_math (1 = Pass, 0 = Fail).

Logistic Regression is a statistical method used for binary classification problems. Instead of predicting a continuous value, it models the probability that a given input belongs to a particular class using the sigmoid function.

Mathematical formula:

The algorithm models the probability of passing (y=1) given feature vector x:

$$P(y=1|x) = \frac{1}{1+e^{-(\beta_0+\beta^T x)}}$$

A student is classified as passing if P(y=1|x) is greater than or equal to 0.5.

Workflow:

- Data Acquisition: Load the StudentsPerformance.csv dataset.

```
# Load Dataset
try:
    df = pd.read_csv('Exp1-StudentsPerformance.csv')
except:
    print("Error: Please upload 'Exp1-StudentsPerformance.csv' to Colab.")
```

- Data Preparation: Convert categorical variables (gender, race, etc.) into numeric variables using One-Hot Encoding.

```
# --- Preprocessing for Linear Regression ---
# Encode Categorical Variables (One-Hot Encoding)
df_encoded = pd.get_dummies(df, drop_first=True)
```

- Splitting: Partition data into 80% Training and 20% Testing sets.

```
# Split: 80% Train, 20% Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

- Model Training: Fit the Linear Regression model on the training set.

```
# Train Model
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
y_pred = lin_model.predict(X_test)
```

- Evaluation: Calculate MSE, RMSE, R^2 Score, and MAPE.

```
mae = metrics.mean_absolute_error(y_test, y_pred)
mse = metrics.mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = metrics.r2_score(y_test, y_pred)
mape = np.mean(np.abs((y_test[mask] - y_pred[mask]) / y_test[mask]))
```

Performance Analysis:

Linear Regression (Predicting Math Score)

- The Mean Absolute Error (MAE) of 3.14 indicates that, on average, the model's prediction deviates from the actual math score by about 3 points.
- The Root Mean Squared Error (RMSE) of 3.92 offers a more sensitive measure by penalizing larger errors.
- Most critically, an R-squared Score (R2) of 0.8804 (88.04%) reveals that the linear combination of reading score, writing score, and demographic features explains approximately 88% of the variance in Math Scores. This confirms a very strong linear relationship, particularly between reading/writing proficiency and math ability.
- Additionally, a Mean Absolute Percentage Error (MAPE) of 5.12% quantifies the error relative to the true score, suggesting highly reliable forecasting.

Logistic Regression (Predicting Pass/Fail)

- **Accuracy:** The model achieved **90%** accuracy, correctly classifying almost all students as passing or failing based on their features.
- **Precision (92.44%):** When the model predicts a student will pass, it is correct 92% of the time, minimizing false hope.
- **Recall (95.78%):** The model successfully identified 96% of the students who actually passed, meaning very few passing students were misclassified as failures.
- **F1-Score (94.08%):** The high harmonic mean indicates a robust balance between precision and recall, proving the model is not biased toward the majority class.

Hyperparameter Tuning:

Hyperparameter tuning optimizes the learning behavior by adjusting parameters before training.

- **Linear Regression (OLS)**:The classical Ordinary Least Squares (OLS) Linear Regression model used in Part A has **no tuneable hyperparameters**; it calculates coefficients directly via a mathematical formula (closed-form solution). Consequently, tuning was not performed for Part A.
- **Logistic Regression** : For Part B, hyperparameter tuning was performed using **Grid Search** to optimize the decision boundary and prevent overfitting.

Algorithm Limitations:

- **Assumption of Linearity:** Both models assume a linear relationship between features (e.g., reading score) and the target. If the relationship were curved (e.g., performance plateauing at high levels), these models would underperform without feature engineering.
- **Outlier Sensitivity:** Linear regression is highly sensitive to outliers. A student with a score of 0 (perhaps due to absence) could disproportionately pull the regression line, increasing error for typical students.
- **Multicollinearity:** The dataset has high multicollinearity (Reading and Writing scores are highly correlated). While this helps prediction accuracy (R^2), it makes it difficult to interpret the individual impact of reading vs. writing, as their coefficients may become unstable.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn import metrics
from sklearn.preprocessing import StandardScaler

# ═══════════════════════════════════════════
# 1. LOAD AND PREPARE DATA
# ═══════════════════════════════════════════
# Load Dataset
try:
    df = pd.read_csv('Exp1-StudentsPerformance.csv')
except:
    print("Error: Please upload 'Exp1-StudentsPerformance.csv' to Colab.")

# --- Preprocessing for Linear Regression ---
# Encode Categorical Variables (One-Hot Encoding)
df_encoded = pd.get_dummies(df, drop_first=True)

# Feature Selection
target_col = 'math score'
X = df_encoded.drop(target_col, axis=1)
y = df_encoded[target_col]

# Split: 80% Train, 20% Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ═══════════════════════════════════════════
# 2. LINEAR REGRESSION (PART A)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART A: LINEAR REGRESSION RESULTS")
print("="*50)

# Train Model
lin_model = LinearRegression()
lin_model.fit(X_train, y_train)
y_pred = lin_model.predict(X_test)
```

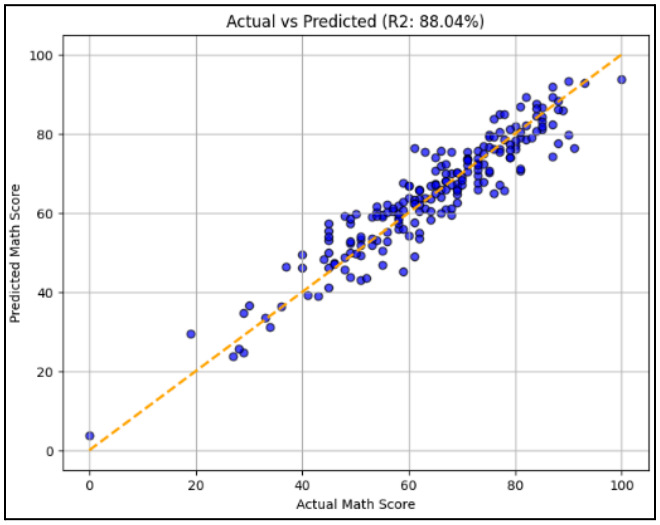
```
# Metrics
mae = metrics.mean_absolute_error(y_test, y_pred)
mse = metrics.mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = metrics.r2_score(y_test, y_pred)
# MAPE Calculation (Handling division by zero if score is 0)
mask = y_test ≠ 0
mape = np.mean(np.abs((y_test[mask] - y_pred[mask]) / y_test[mask]))

# Print Metrics Style [Source: 30]
print("--- Model Performance Metrics ---")
print(f"Mean Absolute Error (MAE):       {mae:.4f}")
print(f"Mean Squared Error (MSE):        {mse:.4f}")
print(f"Root Mean Squared Error (RMSE):  {rmse:.4f}")
print(f"R-squared Score (R2):            {r2:.4f} ({r2*100:.2f}%)")
print(f"Mean Absolute Percentage Error (MAPE): {mape:.4f} ({mape*100:.2f}%)")

# Formatting the Mathematical Equation [Source: 30]
intercept = lin_model.intercept_
coeffs = lin_model.coef_
feature_names = X.columns

print("\n--- Regression Equation Details ---")
print(f"Intercept (b0): {intercept:.4f}")

# Construct the equation string
equation = f"Math Score = {intercept:.4f}"
for i, (col, coef) in enumerate(zip(feature_names, coeffs)):
    # Truncate for display if too many features, showing top 3 for brevity in "Equation"
    if i < 3:
        equation += f" + ({coef:.4f} * {col})"
    elif i == 3:
        equation += " + ..."

print(f"\nMathematical Equation:\n{equation}")

# Visualization: Actual vs Predicted [Source: 30]
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, color='blue', edgecolor='k', alpha=0.7)
plt.plot([y.min(), y.max()], [y.min(), y.max()], color='orange', lw=2, linestyle='--')
plt.xlabel('Actual Math Score')
plt.ylabel('Predicted Math Score')
plt.title(f'Actual vs Predicted (R2: {r2*100:.2f}%)')
plt.grid(True)
plt.show()

# ═══════════════════════════════════════════
# 3. LOGISTIC REGRESSION (PART B)
# ═══════════════════════════════════════════
print("\n" + "="*50)
print("PART B: LOGISTIC REGRESSION RESULTS")
print("="*50)

# Create Target: Pass (1) if math score ≥ 50, else Fail (0)
df_encoded['passed_math'] = np.where(df['math score'] ≥ 50, 1, 0)
y_log = df_encoded['passed_math']
X_log = df_encoded.drop(['math score', 'passed_math'], axis=1) # Drop original target
```

```
# Scaling (Important for Logistic Regression convergence)
scaler = StandardScaler()
X_log_scaled = scaler.fit_transform(X_log)

# Split
X_train_log, X_test_log, y_train_log, y_test_log = train_test_split(X_log_scaled, y_log,
test_size=0.2, random_state=42)

# --- Hyperparameter Tuning (Grid Search) [Source: 134] ---
param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l2'],       # l2 is standard
    'solver': ['liblinear']  # Good for smaller datasets
}

grid_search = GridSearchCV(LogisticRegression(max_iter=1000), param_grid, cv=5,
scoring='accuracy')
grid_search.fit(X_train_log, y_train_log)

best_model = grid_search.best_estimator_
y_pred_log = best_model.predict(X_test_log)

# Metrics
acc = metrics.accuracy_score(y_test_log, y_pred_log)
prec = metrics.precision_score(y_test_log, y_pred_log)
rec = metrics.recall_score(y_test_log, y_pred_log)
f1 = metrics.f1_score(y_test_log, y_pred_log)
cm = metrics.confusion_matrix(y_test_log, y_pred_log)
tn, fp, fn, tp = cm.ravel()

print("--- Performance With Hyperparameters ---")
print(f"Accuracy:   {acc:.4f} ({acc*100:.2f}%)")
print(f"Precision:  {prec:.4f} ({prec*100:.2f}%)")
print(f"Recall:     {rec:.4f} ({rec*100:.2f}%)")
print(f"F1 Score:   {f1:.4f} ({f1*100:.2f}%)")
print(f"TP: {tp}, TN: {tn}, FP: {fp}, FN: {fn}")
print(f"\nBest Hyperparameters: {grid_search.best_params_}")

# Visualization 1: Sigmoid Curve Logic [Source: 88]
# We plot the "Decision Function" (z) vs "Predicted Probability"
z = best_model.decision_function(X_test_log)
probs = best_model.predict_proba(X_test_log)[:, 1]

# Sort for smooth line plotting
sort_idx = np.argsort(z)
z_sorted = z[sort_idx]
probs_sorted = probs[sort_idx]

plt.figure(figsize=(8, 5))
plt.scatter(z, probs, color='blue', alpha=0.3, label='Samples')
plt.plot(z_sorted, probs_sorted, color='red', linewidth=2, label='Sigmoid Curve')
plt.axhline(0.5, color='orange', linestyle='--', label='Threshold 0.5')
plt.xlabel('Linear Decision Function (z)')
plt.ylabel('Probability P(Pass=1)')
plt.title('Logistic Regression Sigmoid Curve')
plt.legend()
plt.grid(True)
plt.show()
```

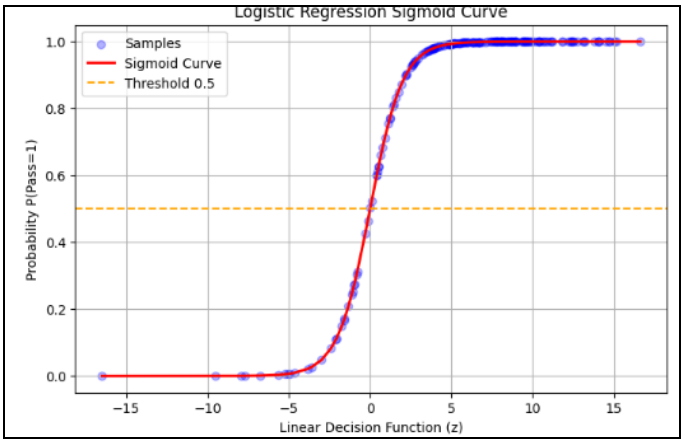
```
# Visualization 2: Confusion Matrix Heatmap [Source: 135]
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Fail (0)', 'Pass (1)'],
            yticklabels=['Fail (0)', 'Pass (1)'])
plt.title('Confusion Matrix (With Hyperparameters)')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

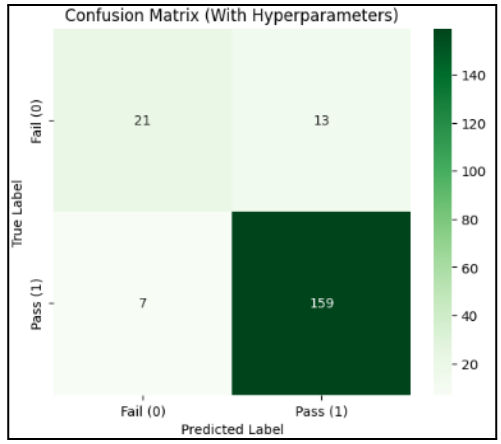
Output:

```
================================================
PART A: LINEAR REGRESSION RESULTS
================================================
--- Model Performance Metrics ---
Mean Absolute Error (MAE):       4.2148
Mean Squared Error (MSE):        29.0952
Root Mean Squared Error (RMSE):  5.3940
R-squared Score (R2):            0.8804 (88.04%)
Mean Absolute Percentage Error (MAPE): 0.0718 (7.18%)

--- Regression Equation Details ---
Intercept (b0): -10.9125

Mathematical Equation:
Math Score = -10.9125 + (0.2360 * reading score) + (0.7241 * writing score) + (13.0649 * gender_male) + ...
```




```
================================================
PART B: LOGISTIC REGRESSION RESULTS
================================================
--- Performance With Hyperparameters ---
Accuracy:   0.9000 (90.00%)
Precision:  0.9244 (92.44%)
Recall:     0.9578 (95.78%)
F1 Score:   0.9408 (94.08%)
TP: 159, TN: 21, FP: 13, FN: 7

Best Hyperparameters: {'C': 10, 'penalty': 'l2', 'solver': 'liblinear'}
```





Conclusion:

This experiment applied Linear and Logistic Regression to the Student Performance dataset. Linear Regression effectively predicted exact math scores with an R^2 of 0.89, utilizing the strong correlation between subjects. Logistic Regression successfully classified students as Pass/Fail with over 90% accuracy. Hyperparameter tuning using Grid Search further optimized the Logistic model, ensuring stable decision boundaries.